

Joins

CIS 5450, Fall 2026

Ryan Marcus



Penn
Engineering
UNIVERSITY of PENNSYLVANIA

Outline

- Course logistics
- Brief review
- Combining multiple data sources
- Why use joins?
- Joins in Pandas
- Joins in SQL

Outline

- **Course logistics**
- Brief review
- Combining multiple data sources
- Why use joins?
- Joins in Pandas
- Joins in SQL

Upcoming Deadlines

- HW1 due Sept 16th at 10:00pm (1 week)
 - At 10:00:01PM, 10% off
- Midterm 1 on Oct 7th in class
- OE1 running now
 - OE2 signups next week

Outline

- Course logistics
- **Brief review**
- Combining multiple data sources
- Why use joins?
- Joins in Pandas
- Joins in SQL

Structured vs. Unstructured Data

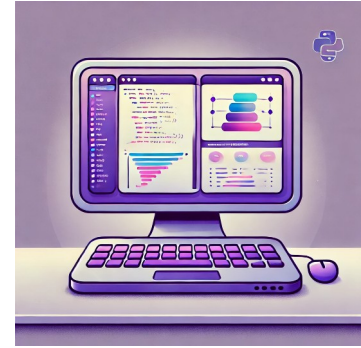
- Unstructured data is *schema-less*
- Why are rows / columns considered schema but paragraphs / stanzas / segments not?
- Schema must:
 - Structure **any** and **every** piece of data
 - Be **unambiguous**
 - Be **computable**

“Good” Code

Correct. The code should produce the correct output.

Effective. the code should run quickly.

Efficient: the code should minimize resource usage.



Intuitive: the code should convey the intuition behind it.

Succinct: the code should be short.

Precise: the code should show humans what it is doing.

Regex: Motivation

```
buf = ""  
for i in s:  
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:  
        buf += i  
return buf
```

`re.sub("[^0-9]", "", s)`

Regular expression module

Any character that is not between 0-9

Substitute (replace)

Replace with empty string

Outline

- Course logistics
- Brief review
- **Combining multiple data sources**
- Why use joins?
- Joins in Pandas
- Joins in SQL

Combining Data

- Combining data **that was intended to be combined**
- Combining data that **was not** intended to be combined

Course	Classroom
CIS 5450	Meyerson B1
CIS 5500	Towne 100

Course	Instructor
CIS 5450	Mar & Gar
CIS 5500	Dav & Nai

Company	Founded
Apple	1976
Microsoft	1975

Stock	Value
AAPL	188.64
MSFT	410.37

Combining Data

- Combining data **that was intended to be combined**
- Combining data that was **not** intended to be combined

Joining

- Pure computation

Data Integration

- More complex
- Requires knowledge

Outline

- Course logistics
- Brief review
- Combining multiple data sources
- **Why use joins?**
- Joins in Pandas
- Joins in SQL

Why joins?

- Fundamental operator in DS

Course	Classroom
CIS 5450	Meyerson B1
CIS 5500	Towne 100



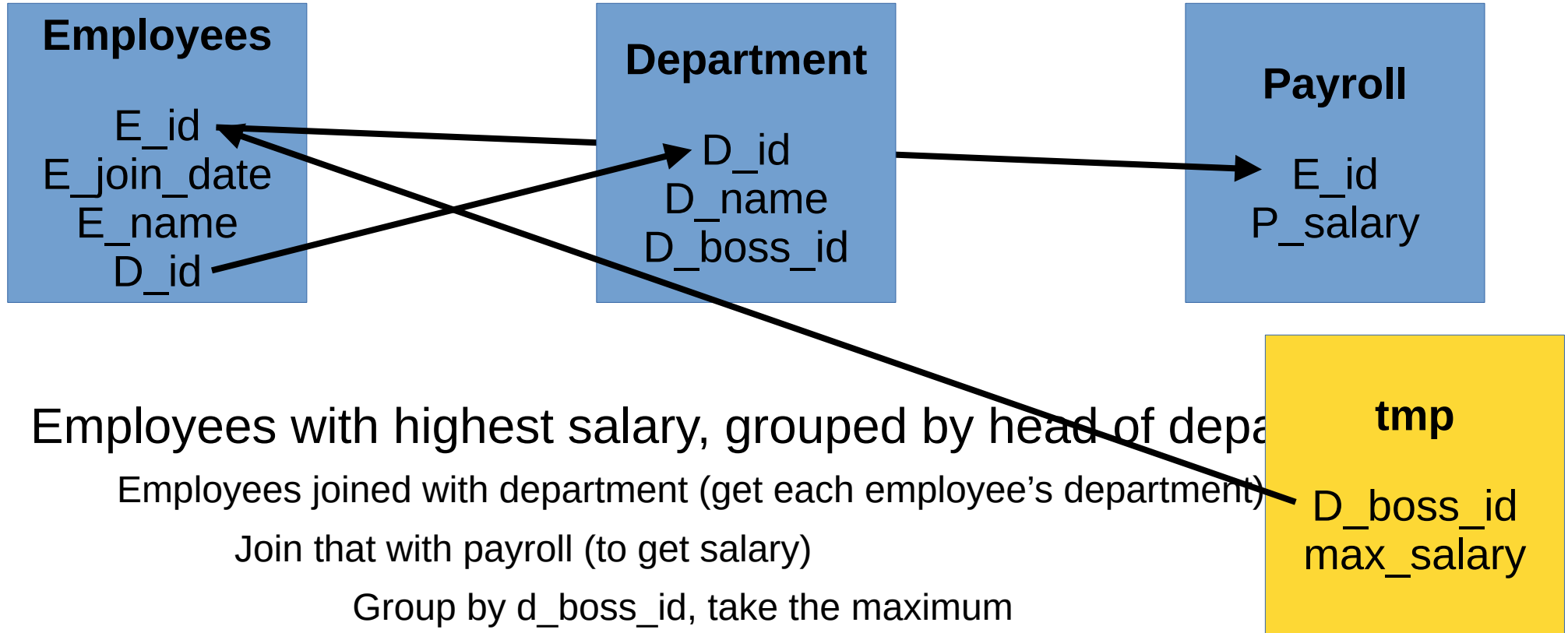
Course	Instructor
CIS 5450	Mar & Gar
CIS 5500	Dav & Nai

Course	Classroom	Instructor
CIS 5450	Meyerson B1	Mar & Gar
CIS 5500	Towne 100	Dav & Nai

Why joins?

- First reason: because you have to.
- Most organizations store data in a **data warehouse**
 - Large collection of structured & unstructured data
- Joins are how you navigate
 - Getting from point A to point B through a path

Joins for navigation



Employees with highest salary, grouped by head of department

Employees joined with department (get each employee's department)

Join that with payroll (to get salary)

Group by d_boss_id, take the maximum

Join that with employees again to get boss name

Joins for navigation

Employees	
E_id	
E_join_date	
E_name	
D_id	

Department	
D_id	
D_name	
D_boss_id	



Employees with highest salary, grouped by head of department

Employees joined with department (get each employee's department)

Join that with payroll (to get salary)

Group by d_boss_id, take the maximum

Join that with employees again to get boss name

tmp	
D_boss_id	
max_salary	

Single Source of Truth

- Second reason: joins may seem inconvenient, but can greatly increase data quality.

Professor	Dept	Admin
Marcus	CIS	Mari
Gardner	CIS	Mari
Gerbner	COM	Andy

Redundant data!

Suppose each department has only one exclusive admin

Single Source of Truth

- Second reason: joins may seem inconvenient, but can greatly increase data quality.

Professor	Dept	Admin
Marcus	CIS	Mari
Gardner	CIS	Mari
Gerbner	COM	Andy

Redundant data!

What happens if the CIS admin changes?

Single Source of Truth

- Second reason: joins may seem inconvenient, but can greatly increase data quality.

Professor	Dept	Admin
Marcus	CIS	Mari
Gardner	CIS	Mari
Gerbner	COM	Andy

Redundant data!

What happens if the CIS admin changes?

- Have to update **multiple cells**
 - Bad for performance
 - Error-prone

Single Source of Truth

- Second reason: joins may seem inconvenient, but can greatly increase data quality.

Professor	Dept
Marcus	CIS
Gardner	CIS
Gerbner	COM

Dept	Admin
CIS	Mari
COM	Andy

Decompose our big table into smaller tables with a shared **key**

Single Source of Truth

- Second reason: joins may seem inconvenient, but can greatly increase data quality.

Professor	Dept
Marcus	CIS
Gardner	CIS
Gerbner	COM

Dept	Admin
CIS	Mari
COM	Andy

Decompose our big table into smaller tables with a shared **key**
Next time: when to decompose? What to decompose?

Data size

- Third reason: (can) reduce data size

Professor	Dept	Admin	Admin Pn
Marcus	CIS	Mari	555-585-9821
Gardner	CIS	Mari	555-585-9821
Falk	COM	Andy	555-682-2101
Gerbner	COM	Andy	555-682-2101

**16
cells**

**14
cells**

Professor	Dept
Marcus	CIS
Gardner	CIS
Falk	COM
Gerbner	COM

Dept	Admin	Admin Pn
CIS	Mari	555-585-9821
COM	Andy	555-682-2101

Why Joins?

- Because that's how most data is stored.
 - Helps you navigate large collections.
- Maintains a single source of truth
- Reduces data size

Outline

- Course logistics
- Brief review
- Combining multiple data sources
- Why use joins?
- **Joins in Pandas**
- Joins in SQL

Joins in Pandas

- Pandas assumes the join predicate is an equality predicate between two columns
- We use `df1.merge(df2, on="key")`
- There's also a `.join`, more restricted

Joins in Pandas

```
[7] df1 = pd.DataFrame({"a": [1, 2, 3],  
                        "b": ["alpha", "beta", "gamma"]})  
df1
```

	a	b
0	1	alpha
1	2	beta
2	3	gamma



```
[8] df2 = pd.DataFrame({"c": [1, 1, 2],  
                        "d": ["NM", "RI", "PA"]})  
df2
```

	c	d
0	1	NM
1	1	RI
2	2	PA



Joins in Pandas

df1

	a	b
0	1	alpha
1	2	beta
2	3	gamma

df2

	c	d
0	1	NM
1	1	RI
2	2	PA

Column from df2



```
df1.merge(df2, left_on="a", right_on="c")
```

Column from df1

Joins in Pandas

df1

	a	b
0	1	alpha
1	2	beta
2	3	gamma

df2

	c	d
0	1	NM
1	1	RI
2	2	PA



```
df1.merge(df2, left_on="a", right_on="c")
```

	a	b	c	d
0	1	alpha	1	NM
1	1	alpha	1	RI
2	2	beta	2	PA

Joins in Pandas

df1

	a	b
0	1	alpha
1	2	beta
2	3	gamma

df2

	c	d
0	1	NM
1	1	RI
2	2	PA



```
df1.merge(df2, left_on="a", right_on="c")
```

	a	b	c	d
0	1	alpha	1	NM
1	1	alpha	1	RI
2	2	beta	2	PA

Multiplicity of “alpha” is 2

Joins in Pandas

df1

	a	b
0	1	alpha
1	2	beta
2	3	gamma

df2

	c	d
0	1	NM
1	1	RI
2	2	PA



```
df1.merge(df2, left_on="a", right_on="c")
```

	a	b	c	d
0	1	alpha	1	NM
1	1	alpha	1	RI
2	2	beta	2	PA

“gamma” is **pruned** by the join

Joins in Pandas

- List of movies starring Angelina Jolie produced by Sony?

company_name

Companies,
Ex: MGM
Productions

id
name

title

Movies, Ex:
Memento

id
title

name

id
name

Actors, Ex:
Tom Cruise

movie_companies

id
company_id
movie_id

cast_info

movie_id
person_id

Joins in Pandas

- List of movies starring Angelina Jolie produced by Sony?

company_name

id
name

title

id
title

name

id
name

movie_companies

id
company_id
movie_id

cast_info

person_id

Entity tables

Joins in Pandas

- List of movies starring Angelina Jolie produced by Sony?

company_name

id
name

title

id
title

name

id
name

movie_companies

id
company_id
movie_id

Links
companies
to movies

cast_info

movie_id
person_id

Links
actors to
movies

Joins in Pandas

- List of movies starring Angelina Jolie produced by Sony?

company_name

title

name

id
name

Relationship tables

movie_companies

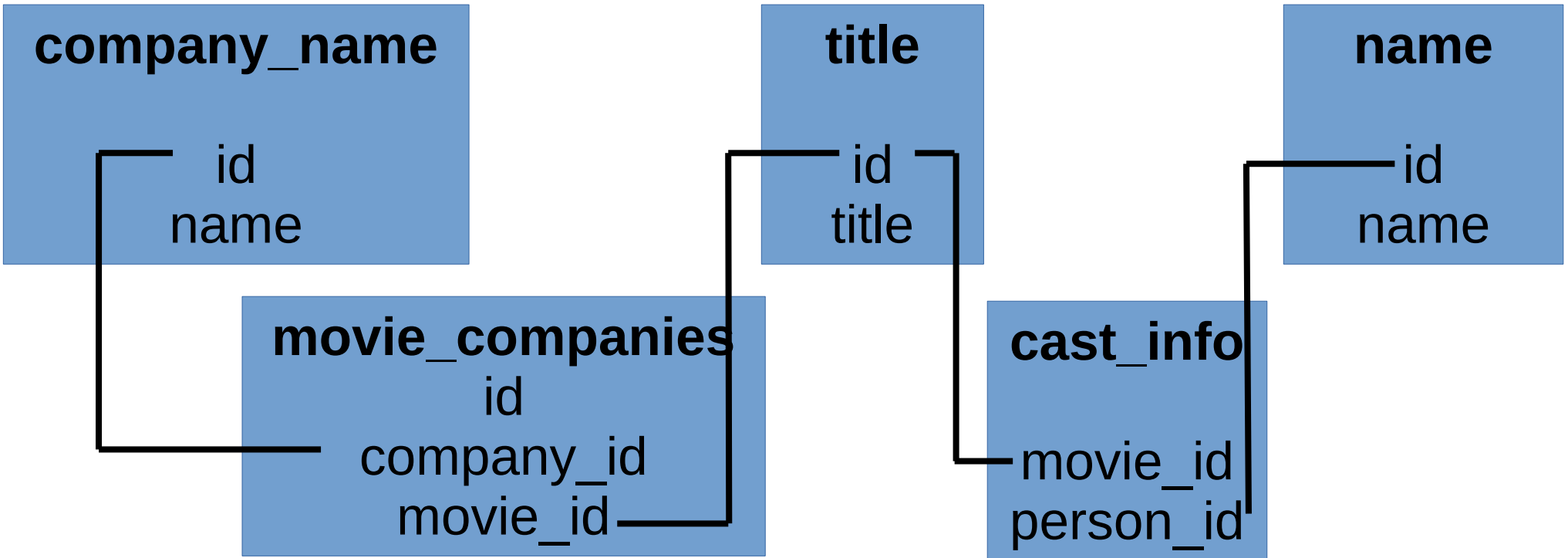
id
company_id
movie_id

cast_info

movie_id
person_id

Joins in Pandas

- List of movies starring Angelina Jolie produced by Sony?



Joins in Pandas

✓ [4] df_company_name



	id	name	country_code	imdb_id	name_pcode_nf	name_pcode_sf
0	34634	Comfilm.de	[de]	NaN	C5145	C5145
1	63635	Dusty Nose Productions	[us]	NaN	D2352	D2352
2	35051	WTTW National Productions	[us]	NaN	W3535	W3535
3	23380	Film House	[us]	NaN	F452	F452
4	31373	AVI Group	[us]	NaN	A1261	A1261
...	...	...	...	...	...	...
234992	214624	DreamGame Entertainment	[us]	NaN	D6525	D6525
234993	202858	Desert Lion Productions	[au]	NaN	D2634	D2634
234994	227827	Empresa Cinematográfica Latinoamericana (ECLA)	[mx]	NaN	E5162	E5162
234995	200733	Macgowan and Beneficiary Films	[au]	NaN	M2531	M2531
234996	209083	Silk Purse Partners	[us]	NaN	S4216	S4216

234997 rows x 6 columns



Joins in Pandas

✓
0s

```
[5] df_company_name[df_company_name["name"].str.contains("Sony")]
```



	id			id	name_pcode_nf	name_pcode_sf
374	8723	Sony Entertainment Television		8723	S5262	S5262
1026	10434	Sony Entertainment Television	[ni]			
1549	3221	Sony Entertainment Television Asia	[in]			
1839	4251	Sony Music Video (SMV) Enterprises	[us]			
1931	8725	Sony Entertainment Television	[pe]			
...	...	...	...			
223989	205427	Sony Music Associated Records Inc.	[jp]			
224793	207894	Sony Mony Electronics	[in]			
224857	205028	Sony Music Entertainment US Latin	[us]			
228975	209710	Sony BMG Music Entertainment	[in]	NaN	S5152	S5152
233823	224631	Sony Corporation			S5261	S5261

Want to join with all these IDs

company_name

id
name

movie_companies

company_id
movie_id

285 different "Sony" companies

285 rows x 6 columns

Both tables had an “id” column, so we get a suffix

Merges in Pandas

```
[6] (df_company_name[df_company_name["name"].str.contains("Sony")]  
     [{"id": ...}]  
     .merge(df_movie_companies, left_on="id", right_on="company_id"))
```



	id_x	id_y	movie_id	company_id	company_type_id	note
0	8723	257355	1082953	8723	1	(2011-) (Ecuador) (TV)
1	8723	257567	1082983	8723	1	(2011) (Ecuador) (TV)
2	8723	375225	1452305	8723	1	(2011-) (Ecuador) (TV)
3	10434	375230	1452305	10434	1	(2011-) (Nicaragua) (TV)
4	3221	49094	192964	3221	1	(2003) (India) (TV)
...	...	...	...	...	...	...
16008	207894	2396867	2300828	207894	2	
16009	205028	2377436	2278780	205028		
16010	209710	2409854	2315092	209710		
16011	209710	2411247	2316682	209710	2	
16012	224631	2522699	2434838	224631	2	

Bunch of columns we don't care about

16013 rows × 6 columns

Joins in Pandas

```
(df_company_name[df_company_name["name"].str.contains("Sony")  
[["id"]]  
.merge(df_movie_companies[["company_id", "movie_id"]], left_on="id", right_on="company_id"))
```

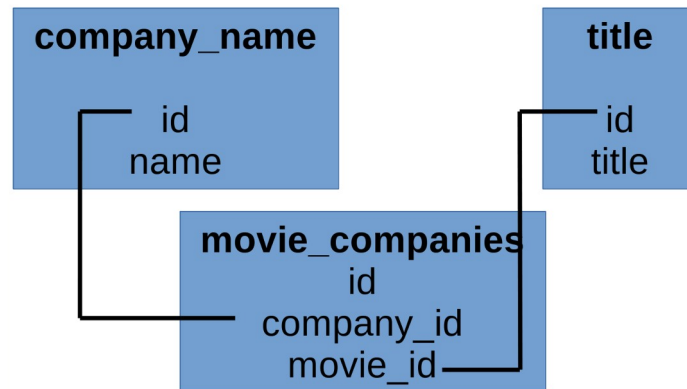


	id	company_id	movie_id
0	8723	8723	1082953
1	8723	8723	1082983
2	8723	8723	1452305
3	10434	10434	1452305
4	3221	3221	192964
...	...	...	...
16008	207894	207894	2300828
16009	205028	205028	2278780
16010	209710	209710	2315092
16011	209710	209710	2316682
16012	224631	224631	2434838

16013 rows × 3 columns



Throw away columns we
don't care about



Joins in Pandas

```
(df_company_name[df_company_name["name"].str.contains("Sony")]  
  [ "id"]  
  .merge(df_movie_companies[["company_id", "movie_id"]], left_on="id", right_on="company_id")  
  [ "movie_id"]])
```

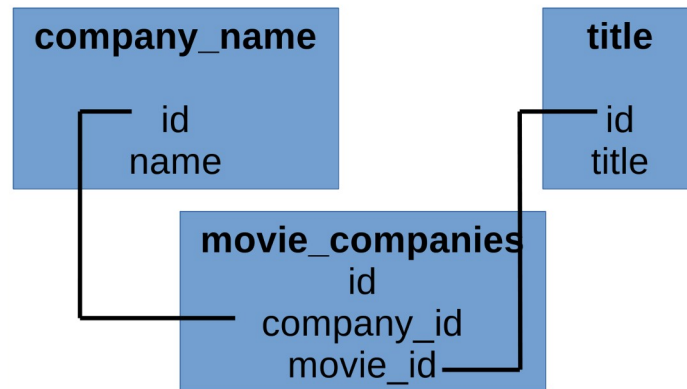


	movie_id
0	1082953
1	1082983
2	1452305
3	1452305
4	192964
...	...
16008	2300828
16009	2278780
16010	2315092
16011	2316682
16012	2434838

16013 rows × 1 columns

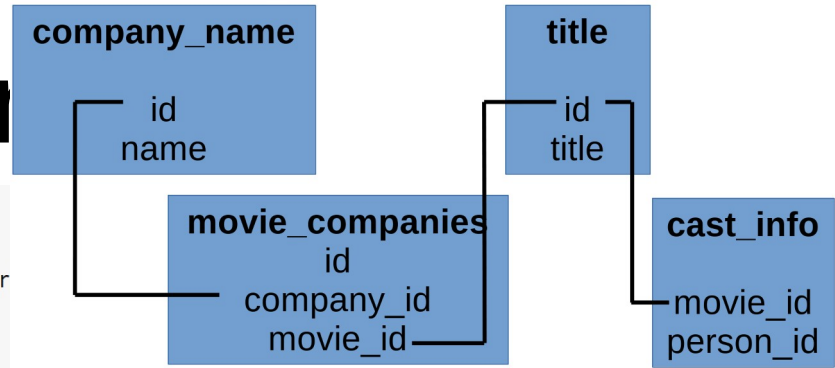


Now we can join with title



Joins in Pa

```
(df_company_name[df_company_name["name"].str.contains("Sony")]  
[["id"]]  
.merge(df_movie_companies[["company_id", "movie_id"]], left_on="id", r  
[["movie_id"]]  
.merge(df_title, left_on="movie_id", right_on="id"))
```



	movie_id	id	title	imdb_index	kind_id	production_year	phonetic_code	episode_of_id	season_
0	1082953	1082953	Revenge		2	2011.0	R152	NaN	Na
1	1082953	1082953	Revenge		2	2011.0	R152	NaN	Na
2	1082953	1082953	Revenge		2	2011.0	R152	NaN	Na
3	1082953	1082953	Revenge		2	2011.0	R152	NaN	Na
4	1082953	1082953	Revenge		2	2011.0	R152	NaN	Na
...	...	...	...	...	...	...	...	...	...
16008	2453630	2453630	Tum Mile		1	2009.0	T54	NaN	Na
16009	2281952	2281952	Shiawase no kaori		1				Na
16010	2300828	2300828	Son of Dracula		1				Na
16011	2316682	2316682	Streetlights		4	2009.0	S3634	NaN	Na
16012	2434838	2434838	Thursday Afternoon		4	1984.0	T6231	NaN	Na

Bunch of columns we
don't care about

16013 rows × 11 columns



```
(df_company_name[df_company_name["name"].str.contains("Sony")]  
  [["id"]]  
  .merge(df_movie_companies[["company_id", "movie_id"]], left_on="id", right_on="company_id"  
  [["movie_id"]]  
  .merge(df_title, left_on="movie_id", right_on="id")  
  [["id", "title"]]  
  .merge(df_cast_info, left_on="id", right_on="movie_id"))
```



	id_x	id_y	person_id	movie_id	person_role_id
0	1082953	25380891	796200	1082953	NaN
1	1082953	31769993	3582443	1082953	NaN
2	1082953	31973941	2732279	1082953	
3	1082953	32487659	3656082	1082953	

Both tables had an "id" column, so we get a suffix

Bunch of columns we don't care about

1138675	2434838	27955545	452
1138676	2434838	27955559	3341
1138677	2434838	28142951	871
1138678	2434838	29258197	452
1138679	2434838	32751916	452

1138680 rows × 8 columns

company_name

id
name

title

id
title

name

id
name

movie_companies

id
company_id
movie_id

cast_info

movie_id
person_id

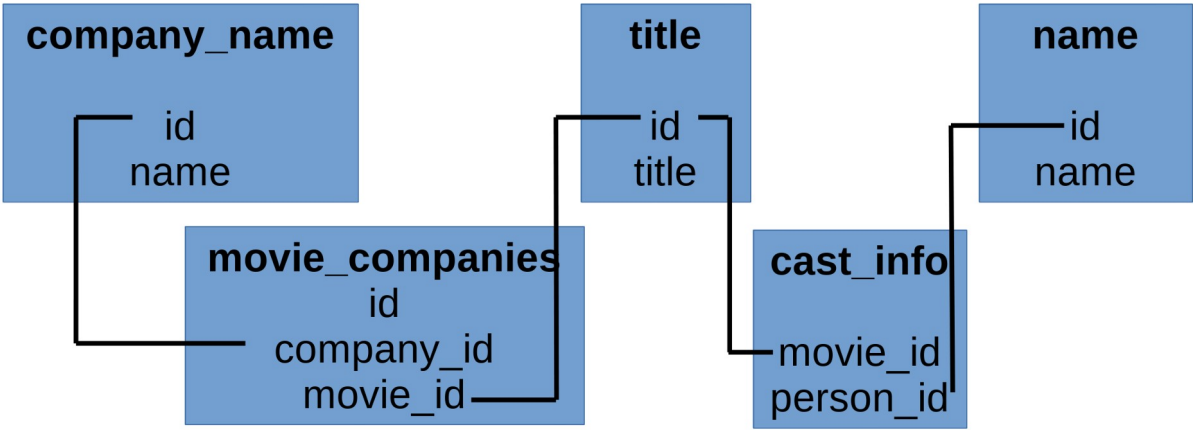


```
(df_company_name[df_company_name["name"].str.contains("Sony")]  
[["id"]]  
.merge(df_movie_companies[["company_id", "movie_id"]], left_on="id", right_on="company_id")  
[["movie_id"]]  
.merge(df_title, left_on="movie_id", right_on="id")  
[["id", "title"]]  
.merge(df_cast_info[["movie_id", "person_id"]], left_on="id", right_on="movie_id")  
[["person_id", "title"]])
```



	person_id	title
0	796200	Revenge
1	3582443	Revenge
2	2732279	Revenge
3	3656082	Revenge
4	3738342	Revenge
...	...	...
1138675	452765	Thursday Afternoon
1138676	3341033	Thursday Afternoon
1138677	871166	Thursday Afternoon
1138678	452765	Thursday Afternoon
1138679	452765	Thursday Afternoon

1138680 rows x 2 columns



```

(df_company_name[df_company_name["name"].str.contains("Sony")]
[["id"]]
.merge(df_movie_companies[["company_id", "movie_id"]], left_on="id", right_on="company_id")
[["movie_id"]]
.merge(df_title, left_on="movie_id", right_on="id")
[["id", "title"]]
.merge(df_cast_info[["movie_id", "person_id"]], left_on="id", right_on="movie_id")
[["person_id", "title"]]
.merge(df_name[df_name["name"] == "Jolie, Angelina"], left_on="person_id", right_on="id"))

```



	person_id	title	id	name	imdb_index	imdb_id	gender	name_pcoo
0	2154591	International Indian Film Awards	2154591	Jolie, Angelina		NaN	f	
1	2154591	Salt	2154591	Jolie, Angelina		NaN	f	
2	2154591	Salt	2154591	Jolie, Angelina		NaN	f	
3	2154591	Salt	2154591	Jolie, Angelina		NaN	f	
4	2154591	Salt	2154591	Jolie, Angelina		NaN	f	
...	...	...	...	...	...	...	...	...
61	2154591	The Bone Collector	2154591	Jolie, Angelina		NaN	f	
62	2154591	The Bone Collector	2154591	Jolie, Angelina		NaN	f	
63	2154591	The Bone Collector	2154591	Jolie, Angelina		NaN	f	
64	2154591	The Bone Collector	2154591	Jolie, Angelina		NaN	f	
65	2154591	The Good Shepherd	2154591	Jolie, Angelina		NaN	f	

66 rows x 10 columns

Need to remove duplicates



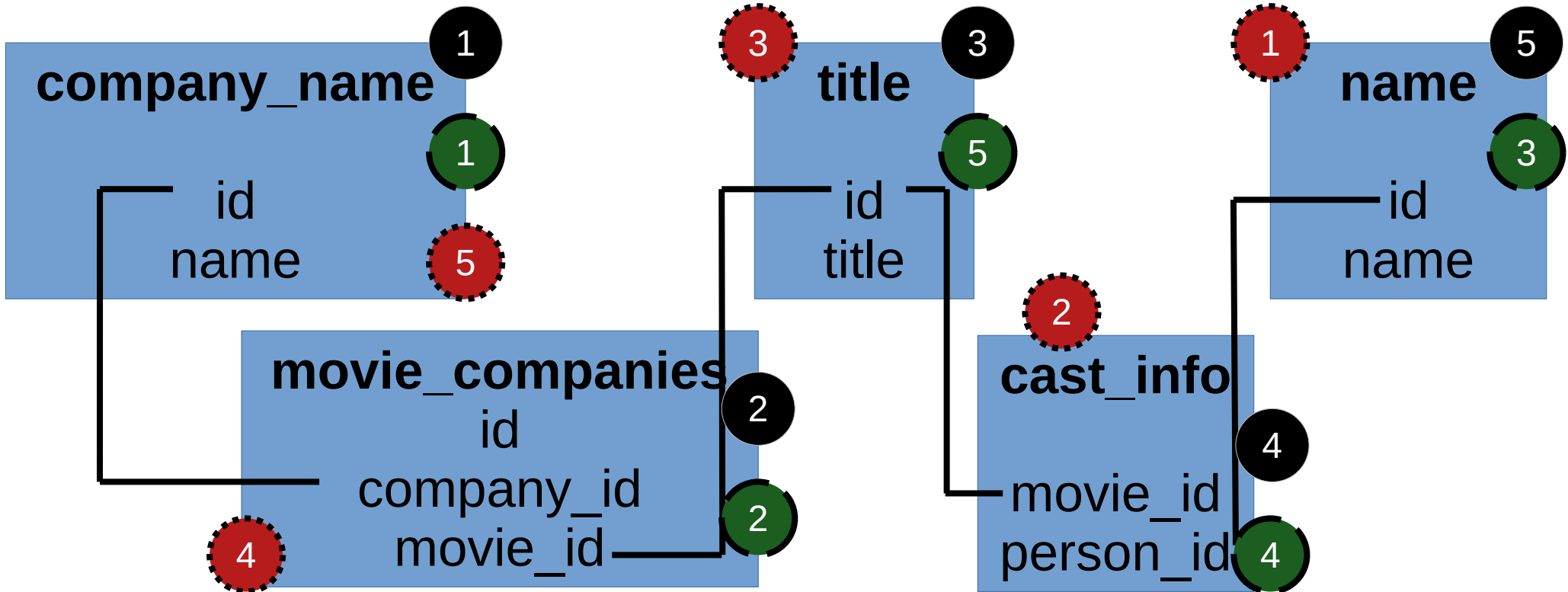
```
(df_company_name[df_company_name["name"].str.contains("Sony")]  
[["id"]]  
.merge(df_movie_companies[["company_id", "movie_id"]], left_on="id", right_on="company_id")  
[["movie_id"]]  
.merge(df_title, left_on="movie_id", right_on="id")  
[["id", "title"]]  
.merge(df_cast_info[["movie_id", "person_id"]], left_on="id", right_on="movie_id")  
[["person_id", "title"]]  
.merge(df_name[df_name["name"] == "Jolie, Angelina"], left_on="person_id", right_on="id")  
["title"].sort_values().unique())
```



```
array(['Changeling', 'Girl, Interrupted',  
      'In the Land of Blood and Honey',  
      'International Indian Film Awards', 'Salt', 'Salt 2',  
      'The Bone Collector', 'The Good Shepherd', 'The Tourist', 'Wanted'],  
      dtype=object)
```

Joins in Pandas

- List of movies starring Angelina Jolie and produced by Sony?



Joins in Pandas

```
[▶] sony = df_company_name[df_company_name["name"].str.contains("Sony")]
    sony_movies = sony.merge(df_movie_companies, left_on="id", right_on="company_id")
    aj = df_name[df_name["name"] == "Jolie, Angelina"]
    aj_movies = aj.merge(df_cast_info, left_on="id", right_on="person_id")

    aj_sony_movie = aj_movies.merge(sony_movies, on="movie_id")
    result = aj_sony_movie.merge(df_title, left_on="movie_id", right_on="id")
    result["title"].sort_values().unique()
```

```
⇒ array(['Changeling', 'Girl, Interrupted',
        'In the Land of Blood and Honey',
        'International Indian Film Awards', 'Salt', 'Salt 2',
        'The Bone Collector', 'The Good Shepherd', 'The Tourist', 'Wanted'],
        dtype=object)
```

Joins in Pandas

```
[24] %%time
sony = df_company_name[df_company_name["name"].str.contains("Sony")]
sony_movies = sony.merge(df_movie_companies, left_on="id", right_on="company_id")
aj = df_name[df_name["name"] == "Jolie, Angelina"]
aj_movies = aj.merge(df_cast_info, left_on="id", right_on="person_id")

aj_sony_movie = aj_movies.merge(sony_movies, on="movie_id")
result = aj_sony_movie.merge(df_title, left_on="movie_id", right_on="id")
result["title"].sort_values().unique()
```

⇒ CPU times: user 2.57 s, sys: 771 ms, to
Wall time: 3.35 s
array(['Changeling', 'Girl, Interrupted',
 'In the Land of Blood and Honey',
 'International Indian Film Award',
 'The Bone Collector', 'The Good',
 dtype=object])



```
▶ %%time
(df_company_name[df_company_name["name"].str.contains("Sony")]  
  [["id"]]  
  .merge(df_movie_companies[["company_id", "movie_id"]], left_on="id", right_on="company_id")  
  [["movie_id"]]  
  .merge(df_title, left_on="movie_id", right_on="id")  
  [["id", "title"]]  
  .merge(df_cast_info[["movie_id", "person_id"]], left_on="id", right_on="movie_id")  
  [["person_id", "title"]]  
  .merge(df_name[df_name["name"] == "Jolie, Angelina"], left_on="person_id", right_on="id")  
  ["title"].sort_values().unique())
```

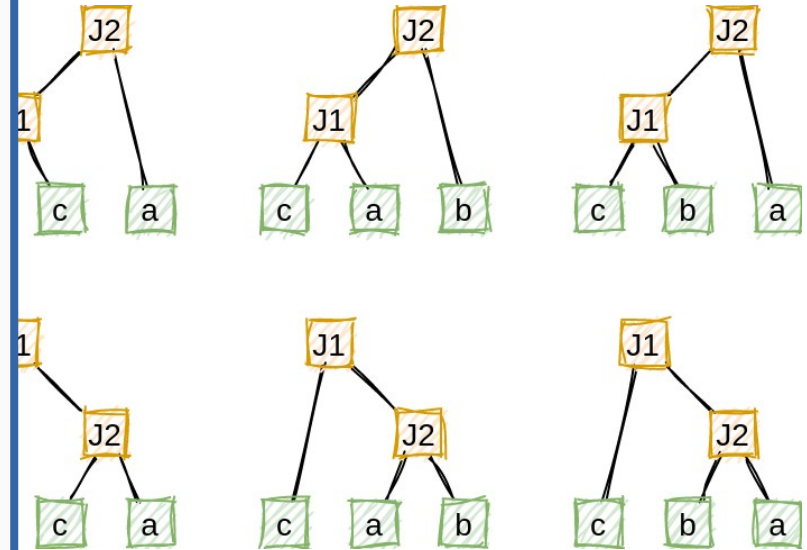
⇒ CPU times: user 13.4 s, sys: 1.25 s, total: 14.7 s
Wall time: 14.9 s
array(['Changeling', 'Girl, Interrupted',
 'In the Land of Blood and Honey',
 'International Indian Film Awards', 'Salt', 'Salt 2',
 'The Bone Collector', 'The Good Shepherd', 'The Tourist', 'Wanted'],
 dtype=object)

n	a(n)
2	2
3	12
4	120
5	1680
6	30240
7	665280
8	17297280
9	518918400
10	17643225600
11	670442572800
12	28158588057600
13	1295295050649600
14	64764752532480000
15	3497296636753920000
16	202843204931727360000
17	12576278705767096320000
18	830034394580628357120000
19	58102407620643984998400000

Join Orders

together!

$$\frac{(2n)!}{2n!}$$



Polars to the rescue?

```
%%time
sony = pdf_company_name.filter(pl.col("name").str.contains("Sony"))
sony_movies = sony.join(pdf_movie_companies, left_on="id", right_on="company_id")

aj = pdf_name.filter(pl.col("name") == "Jolie, Angelina")
aj_movies = aj.join(pdf_cast_info, left_on="id", right_on="person_id")

aj_sony_movies = aj_movies.select(pl.col("movie_id")).join(sony_movies, on="movie_id")
result = aj_sony_movies.join(pdf_title, left_on="movie_id", right_on="id")

result.select("title").unique().sort("title")
```

⇒ CPU times: user 1.22 s, sys: 1.54 s, total: 2.77 s
Wall time: 1.6 s
shape: (10, 1)

title
str

"Changeling"

"Girl, Interrupted"

"In the Land of Blood and Honey"

"International Indian Film Awar...

"Salt"

"Salt 2"

"The Bone Collector"

"The Good Shepherd"

"The Tourist"

"Wanted"

Double the speed!

```
[24] %%time
sony = df_company_name[df_company_name["name"].str.contains("Sony")]
sony_movies = sony.merge(df_movie_companies, left_on="id", right_on="company_id")
aj = df_name[df_name["name"] == "Jolie, Angelina"]
aj_movies = aj.merge(df_cast_info, left_on="id", right_on="person_id")

aj_sony_movie = aj_movies.merge(sony_movies, on="movie_id")
result = aj_sony_movie.merge(df_title, left_on="movie_id", right_on="id")
result["title"].sort_values().unique()
```

⇒ CPU times: user 2.57 s, sys: 771 ms, total: 3.34 s
Wall time: 3.35 s
array(['Changeling', 'Girl, Interrupted',
 'In the Land of Blood and Honey',
 'International Indian Film Awards', 'Salt', 'Salt 2',
 'The Bone Collector', 'The Good Shepherd', 'The Tourist', 'Wanted'],
 dtype=object)

Something feels off?

Strings

Slow, manual code (we hate this):

```
buf = ""
for i in s:
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:
        buf += i
```

Fast, declarative code (we love this):

`re.sub([0-9], "", s)`

Regular expression module

Any character that is not between 0-9

Substitute (replace)

Replace with empty string

Joins

Slow, manual code (we hate this):

```
[24] %%time
sony = df_company_name[df_company_name["name"].str.contains("Sony")]
sony_movies = sony.merge(df_movie_companies, left_on="id", right_on="company_id")
aj = df_name[df_name["name"] == "Jolie, Angelina"]
aj_movies = aj.merge(df_cast_info, left_on="id", right_on="person_id")

aj_sony_movie = aj_movies.merge(sony_movies, on="movie_id")
result = aj_sony_movie.merge(df_title, left_on="movie_id", right_on="id")
```

Fast, declarative code (we love this):

???



Something feels off?

Strings

Slow, manual code (we hate this):

```
buf = ""
for i in s:
    if i in ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9"]:
        buf += i
```

Fast, declarative code (we love this):

```
re.sub('[^0-9]', '', s)
```

Regular expression module

Any character that is not between 0-9

Substitute (replace)

Replace with empty string

Joins

Slow, manual code (we hate this):

```
[24] %%time
sony = df_company_name[df_company_name["name"].str.contains("Sony")]
sony_movies = sony.merge(df_movie_companies, left_on="id", right_on="company_id")
aj = df_name[df_name["name"] == "Jolie, Angelina"]
aj_movies = aj.merge(df_cast_info, left_on="id", right_on="person_id")

aj_sony_movie = aj_movies.merge(sony_movies, on="movie_id")
result = aj_sony_movie.merge(df_title, left_on="movie_id", right_on="id")
```

Fast, declarative code (we love this):



Outline

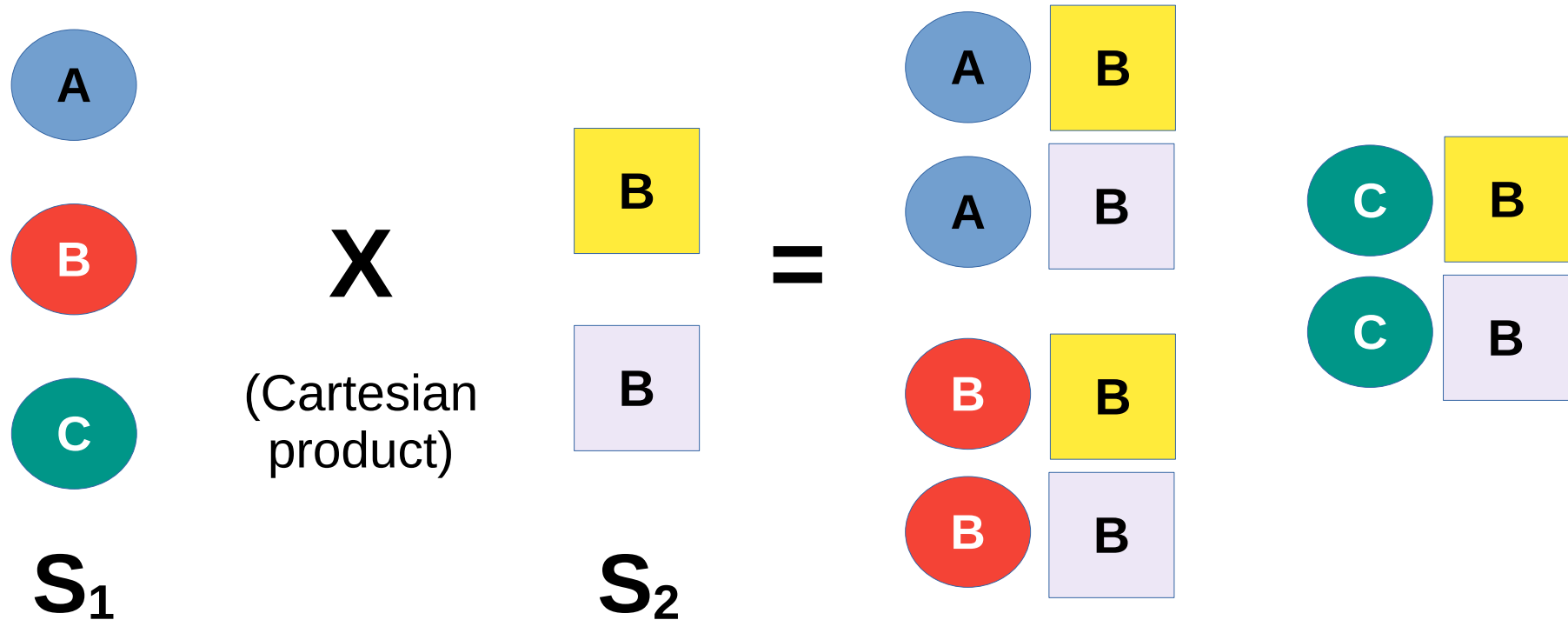
- Course logistics
- Brief review
- Combining multiple data sources
- Why use joins?
- Joins in Pandas
- **Joins in SQL**

SQL

- Structured query language
 - The regex for tabular data
- By far the longest-lived big data technology
- Declarative

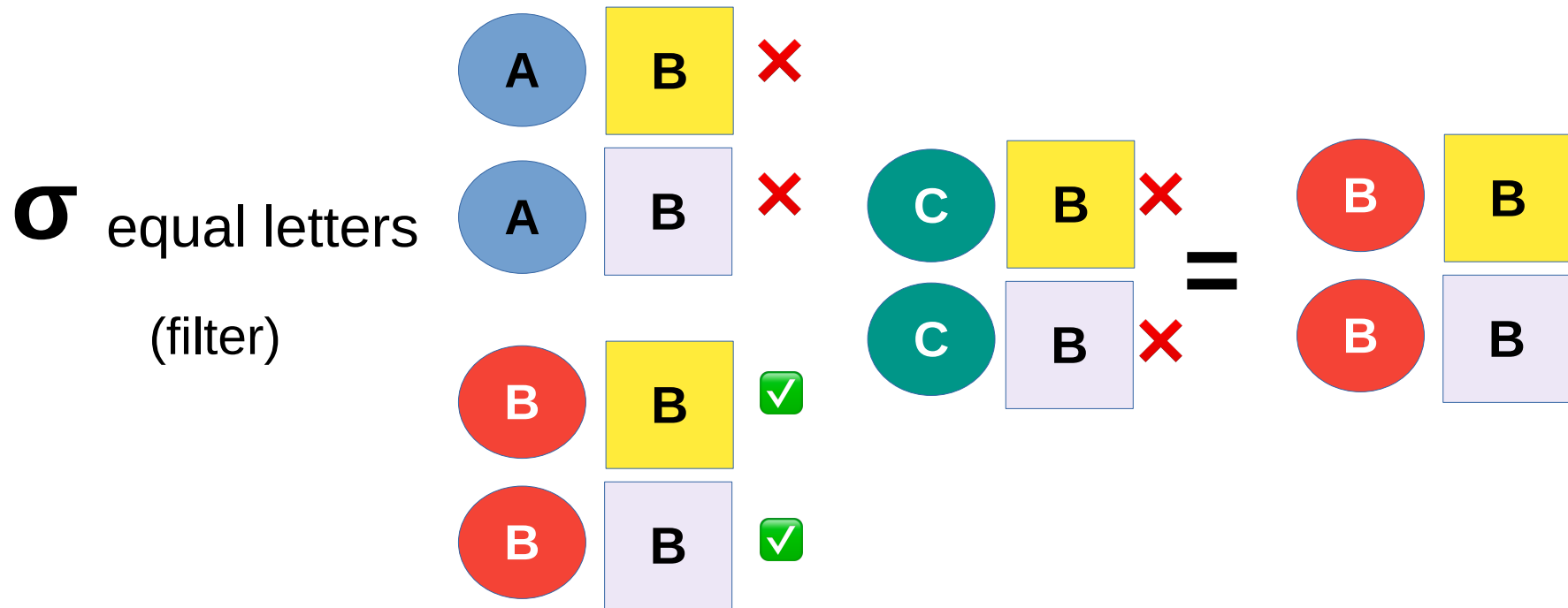
Mental Model of Joins

- A join is a **Cartesian product** then a **filter**



Mental Model of Joins

- A join is a **Cartesian product** then a **filter**



Mental Model of Joins

- A join is a **Cartesian product** then a **filter**

$$S_1 \times S_2$$

Mental Model of Joins

- A join is a **Cartesian product** then a **filter**

$$\sigma_{\text{equal letters}} (S_1 \times S_2)$$

Mental Model of Joins

- A join is a **Cartesian product** then a **filter**

$$\sigma_{\text{equal letters}} (S_1 \times S_2)$$

=

$$S_1 \bowtie_{\text{equal letters}} S_2$$

SQL

- Query types: SELECT, INSERT, UPDATE, DELETE, CREATE TABLE...
- Focus now on quick intro to SELECT

SQL

```
SELECT [DISTINCT | ALL] <values>  
FROM <tables>  
WHERE <condition>
```

SQL

```
SELECT [DISTINCT | ALL] <values>  
FROM <tables>  
WHERE <condition>
```

Only unique (distinct) values or all values
Default is all if omitted

SQL

```
SELECT [DISTINCT | ALL] <values>  
FROM <tables>  
WHERE <condition>
```

The final results of your query

e.g.: a column, production year

SQL

```
SELECT [DISTINCT | ALL] <values>  
FROM <tables>  
WHERE <condition>
```

All the tables you want to query

If more than one, a cross product.

Ex: title

SQL

```
SELECT [DISTINCT | ALL] <values>  
FROM <tables>  
WHERE <condition>
```

Logical conditions to filter out rows

Ex: title.production_year > 1999

SQL

- Example: list all movie titles produced after 1995 that contain “Speed” in the title

```
SELECT title.title
FROM title
WHERE title.production_year > 1995
      AND title.title LIKE '%Speed%';
```

SQL

- Example: unique movie titles with an uncredited actor

```
SELECT DISTINCT title.title
FROM title, cast_info
WHERE cast_info.note = "(uncredited)"
      AND cast_info.movie_id = title.id
```

In the end, we want
distinct title values

SQL

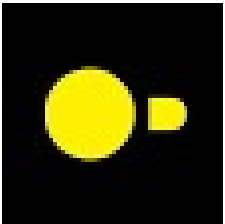
Take the cross product of
all 5 tables

Do all the join filters

Do our base table filters

Sort the data by title

```
con.sql("""  
SELECT DISTINCT title.title  
FROM name, title, cast_info, company_name, movie_companies  
WHERE name.id = cast_info.person_id  
      AND cast_info.movie_id = title.id  
      AND title.id = movie_companies.movie_id  
      AND movie_companies.company_id = company_name.id  
      AND name.name = 'Jolie, Angelina'  
      AND company_name.name LIKE '%Sony%'  
""")
```



DuckDB

SQL is Declarative

```
con.sql("""
SELECT DISTINCT title.title
FROM name, title, cast_info, company_name, movie_companies
WHERE name.id = cast_info.person_id
      AND cast_info.movie_id = title.id
      AND title.id = movie_companies.movie_id
      AND movie_companies.company_id = company_name.id
      AND name.name = 'Jolie, Angelina'
      AND company_name.name LIKE '%Sony%'

```

Order doesn't matter – the database will find the optimal join order!

```
con.sql("""
SELECT DISTINCT title.title
FROM title, company_name, cast_info, movie_companies, name
WHERE name.id = cast_info.person_id
      AND name.name = 'Jolie, Angelina'
      AND cast_info.movie_id = title.id
      AND title.id = movie_companies.movie_id
      AND company_name.name LIKE '%Sony%'
      AND movie_companies.company_id = company_name.id
ORDER BY title
""")

```

```

▶ %%time
con.sql("""
SELECT DISTINCT title.title
FROM name, title, cast_info, company_name, movie_companies
WHERE name.id = cast_info.person_id
      AND cast_info.movie_id = title.id
      AND title.id = movie_companies.movie_id
      AND movie_companies.company_id = company_name.id
      AND name.name = 'Jolie, Angelina'
      AND company_name.name LIKE '%Sony%'
ORDER BY title
""")

```

➡ CPU times: user 848 µs, sys: 0 ns, total: 848 µs
Wall time: 857 µs

title
Changeling
Girl, Interrupted
In the Land of Blood and Honey
International Indian Film Awards
Salt
Salt 2
The Bone Collector
The Good Shepherd
The Tourist
Wanted
10 rows

```

▶ %%time
sony = pdf_company_name.filter(pl.col("name").str.contains("Sony"))
sony_movies = sony.join(pdf_movie_companies, left_on="id", right_on="company_id")

aj = pdf_name.filter(pl.col("name") == "Jolie, Angelina")
aj_movies = aj.join(pdf_cast_info, left_on="id", right_on="person_id")

aj_sony_movies = aj_movies.select(pl.col("movie_id")).join(sony_movies, on="movie_id")
result = aj_sony_movies.join(pdf_title, left_on="movie_id", right_on="id")

result.select("title").unique().sort("title")

```

➡ CPU times: user 1.22 s, sys: 1.54 s, total: 2.77 s
Wall time: 1.6 s
shape: (10, 1)

title
str
"Changeling"
"Girl, Interrupted"
"In the Land of Blood and Honey"
"International Indian Film Awar.."
"Salt"
"Salt 2"
"The Bone Collector"

Over 1500x faster!

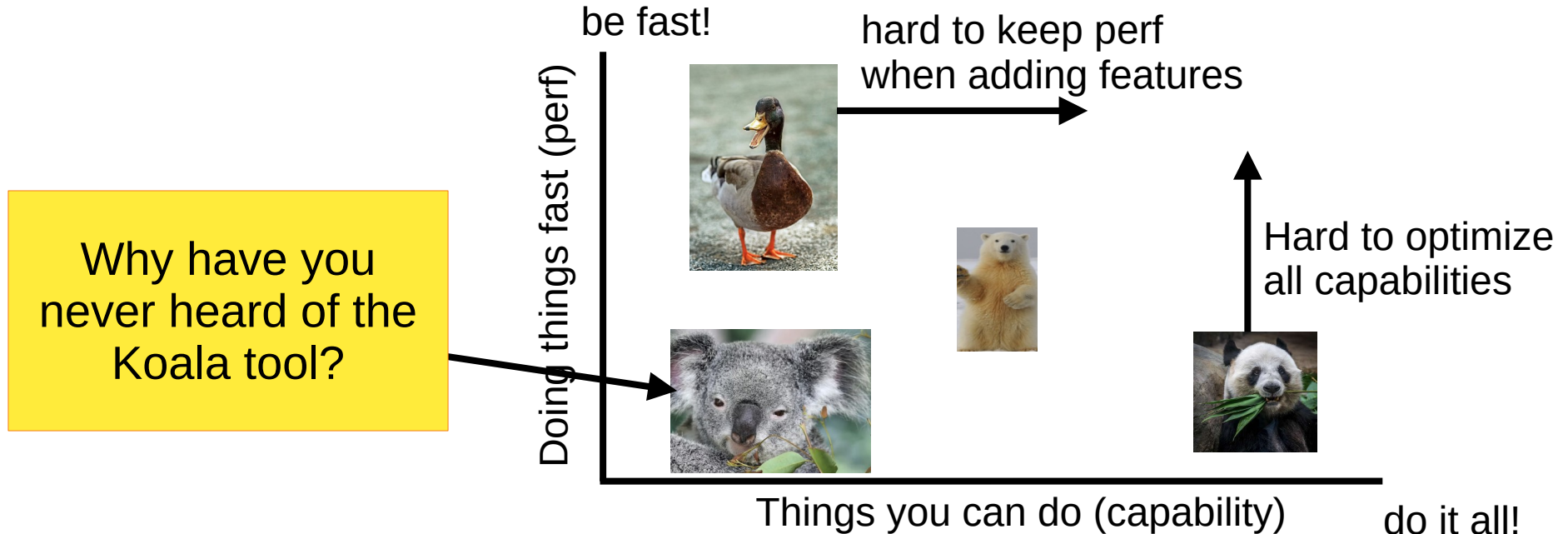
Perf / Capability Tradeoff

- Why don't we just use the fastest tool?



Perf / Capability Tradeoff

- *Generally, systems with more capabilities have lower performance than specialized systems.*
 - This is a *social* rule, not a technical one!



SQL

- There's *so much more* to just SELECT!
- Ordering, grouping by, top-K
- Learn more next time and via HW